



Leche
para el bienestar

MANUAL TÉCNICO

Sistema de inventarios de leche en polvo GEO

Versión 1.0 | Junio 2025

Índice

1. Introducción.....	3
1.1 Propósito del manual.....	3
1.2 Alcance del documento.....	3
1.3 Resumen general del sistema.....	3
2. Arquitectura del sistema.....	4
2.1 Cómo está compuesto el proyecto.....	4
2.2 Patrones de diseño.....	4
2.3 Tecnologías utilizadas.....	5
3. Base de datos.....	6
3.1 SQLite (base de datos local).....	6
3.2 Sincronización con Firebird.....	6
3.3 Respaldo y restauración.....	6
4. Usuarios y accesos.....	7
4.1 Roles y permisos (RBAC).....	7
4.2 Alta y baja de usuarios.....	7
4.3 Contraseñas.....	9
5. Despliegue en la red institucional.....	9
5.1 Infraestructura (Windows Server 2012, VMware, Ubuntu Server).....	9
5.2 Configuración de red (NAT y port forwarding).....	10
6. Instalación del sistema.....	11
6.1 Requisitos previos.....	11
6.2 Clonar el repositorio (Git).....	12
6.3 Configuración del entorno.....	12
6.4 Construir y levantar con Docker.....	13
6.5 Verificación de la instalación.....	13
7. Mantenimiento.....	14
7.1 Monitoreo y bitácora de errores.....	14
7.2 Cómo aplicar actualizaciones.....	14
7.3 Respallos periódicos.....	14
7.4 Solución de problemas comunes.....	15
8. Anexos.....	15
8.1 Estructura de carpetas del proyecto.....	15

1. Introducción

1.1 Propósito del manual

Este manual técnico documenta la arquitectura, la base de datos, la gestión de usuarios, el despliegue y el mantenimiento del sistema de inventarios de leche en polvo. Su propósito es que cualquier persona con perfil técnico pueda instalar, desplegar o dar mantenimiento al sistema sin depender exclusivamente del conocimiento del autor original, y sin tener que descubrir por prueba y error decisiones que ya están documentadas aquí.

1.2 Alcance del documento

Este manual cubre exclusivamente los aspectos técnicos del sistema: arquitectura, base de datos, accesos, despliegue, instalación y mantenimiento. No cubre el uso cotidiano de la aplicación desde la perspectiva del promotor, el supervisor o el área de distribución; para ello existe un Manual de Usuario independiente, orientado a quien opera el sistema día a día y no a quien lo instala o le da soporte.

1.3 Resumen general del sistema

El sistema digitaliza el proceso de inventario y distribución de leche en polvo de Leche del Bienestar en Oaxaca, sustituyendo formatos físicos y hojas de cálculo dispersas por una aplicación web con cuatro módulos (administrador, promotor, supervisor y distribución), desarrollada en PHP, con interfaz basada en Material Design 3, persistencia local en SQLite, sincronización periódica con el sistema heredado Firebird, generación de documentos PDF mediante FPDF, y desplegada mediante un contenedor Docker.

2. Arquitectura del sistema

2.1 Cómo está compuesto el proyecto

El proyecto se organiza por módulo de usuario y por responsabilidad técnica. La siguiente tabla resume las carpetas principales del repositorio:

Carpeta	Contenido
admin/	Módulo de administración: usuarios, errores y sincronización.
distribucion/	Módulo del Departamento de Distribución.
promotores/	Módulo del actor Promotor.
supervisor/	Módulo del actor Supervisor.
src/Controlador, src/Servicio, src/Repositorio	Lógica de negocio y acceso a datos.
database/	Base de datos Firebird heredada y su driver de conexión.
datos/	Base de datos SQLite local y documentos generados por cada módulo.
fpdf/	Librería de generación de documentos PDF.

2.2 Patrones de diseño

El sistema sigue una arquitectura en capas: presentación (vistas PHP con Material Design 3, separadas por módulo), lógica de negocio (controladores y servicios), acceso a datos (patrón Repositorio) y persistencia (SQLite y Firebird). Esta separación permite mantener cada capa de forma independiente: un cambio en la interfaz no debería requerir tocar la lógica de negocio, y un cambio en cómo se almacenan los datos no debería requerir tocar las vistas.

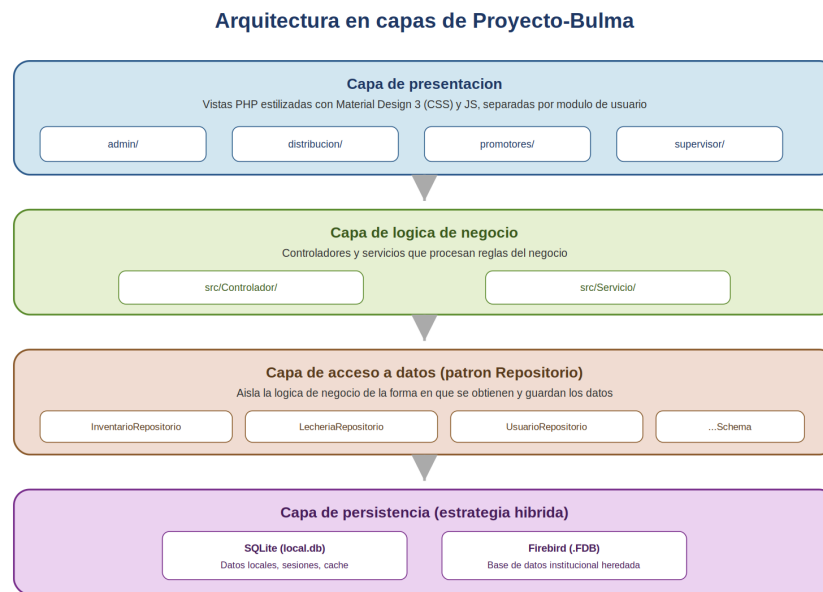


Figura 1. Arquitectura en capas del sistema.

2.3 Tecnologías utilizadas

Tecnología	Uso en el sistema
PHP	Lenguaje del backend; procesa la lógica de los cuatro módulos.
Material Design 3	Lenguaje visual de la interfaz, adaptado a dispositivos móviles para el trabajo en campo.
SQLite	Base de datos local; almacena toda la operación diaria del sistema.
Firebird + driver Jaybird	Sistema heredado institucional, utilizado únicamente para sincronizar el padrón de beneficiarios.
FPDF	Generación de comprobantes en PDF (inventarios, reportes, requerimientos).
Docker	Empaquetado y despliegue del entorno de ejecución completo.
Git / GitHub	Control de versiones del código fuente.

3. Base de datos

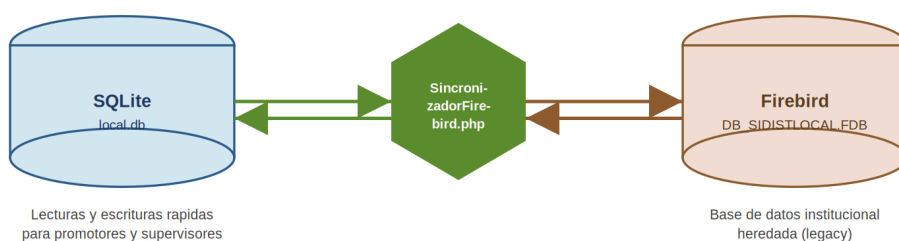
3.1 SQLite (base de datos local)

Toda la operación diaria del sistema (inventarios, reportes, requerimientos, usuarios y sesiones) se lee y se escribe sobre un único archivo: `datos/local.db`. Este archivo concentra la base de datos completa del sistema y debe respaldarse con regularidad (ver sección 3.3).

3.2 Sincronización con Firebird

Firebird (archivo `database/DB_SIDISTLOCAL.FDB`) es el sistema heredado institucional. El sistema no consulta Firebird en vivo durante la operación normal de los módulos; se utiliza únicamente para sincronizar el padrón de beneficiarios cuando oficinas centrales reportan cambios (altas, bajas, apertura o cierre de lecherías). Esta sincronización corre a través del servicio `src/Servicio/SincronizadorFirebird.php`, utilizando el driver Jaybird ubicado en `database/Driver/jaybird-jdk18-3.0.12.jar`.

Estrategia híbrida de datos: SQLite y Firebird



Elaboración propia con base en la estructura real del repositorio Proyecto-Bulma

Figura 2. Estrategia de persistencia híbrida (SQLite y Firebird).

3.3 Respaldo y restauración

Dado que SQLite almacena toda la base de datos en un único archivo, el respaldo consiste en copiar dicho archivo a una ubicación segura:

```
cp datos/local.db respaldos/local_$(date +%F).db
```

Se recomienda automatizar este comando con una tarea programada (cron) de frecuencia diaria, y conservar copias de al menos los últimos 30 días.

4. Usuarios y accesos

4.1 Roles y permisos (RBAC)

El sistema implementa control de acceso basado en roles: cada usuario pertenece a uno de cuatro roles (administrador, promotor, supervisor o distribución) y solo puede acceder a los módulos correspondientes a su rol. Esta validación se aplica en cada petición mediante `admin/guard.php` e `includes/session_guard.php`.

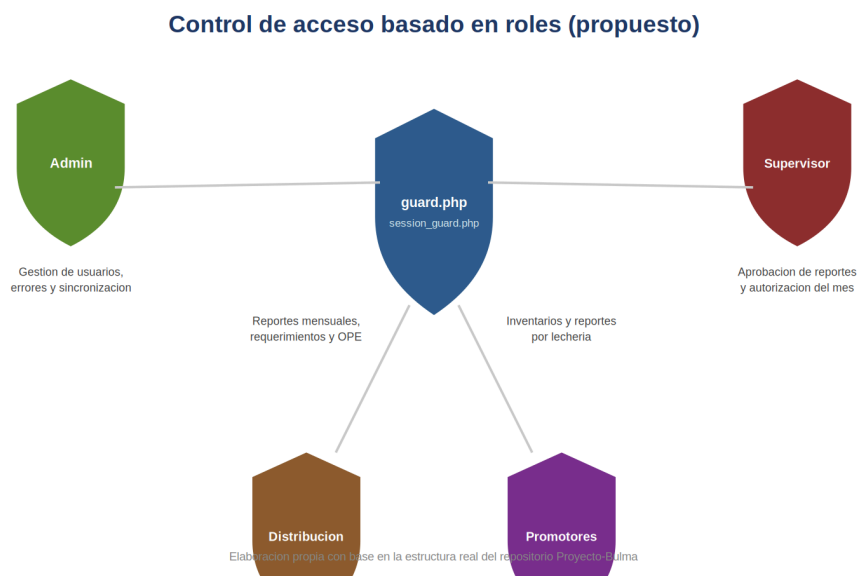


Figura 3. Control de acceso basado en roles (RBAC).

4.2 Alta y baja de usuarios

El alta, baja y edición de usuarios normales (promotores, supervisores, personal de distribución) se realiza desde el módulo de administración, a través de `admin/usuarios.php` y su endpoint `admin/api_usuarios.php`.

El acceso al panel de administración no se realiza mediante el inicio de sesión convencional de los otros módulos, sino mediante un token de seguridad incluido como

parámetro en la URL, gestionado por `admin/cli_token.php`. El panel se accede con una dirección con la siguiente forma:

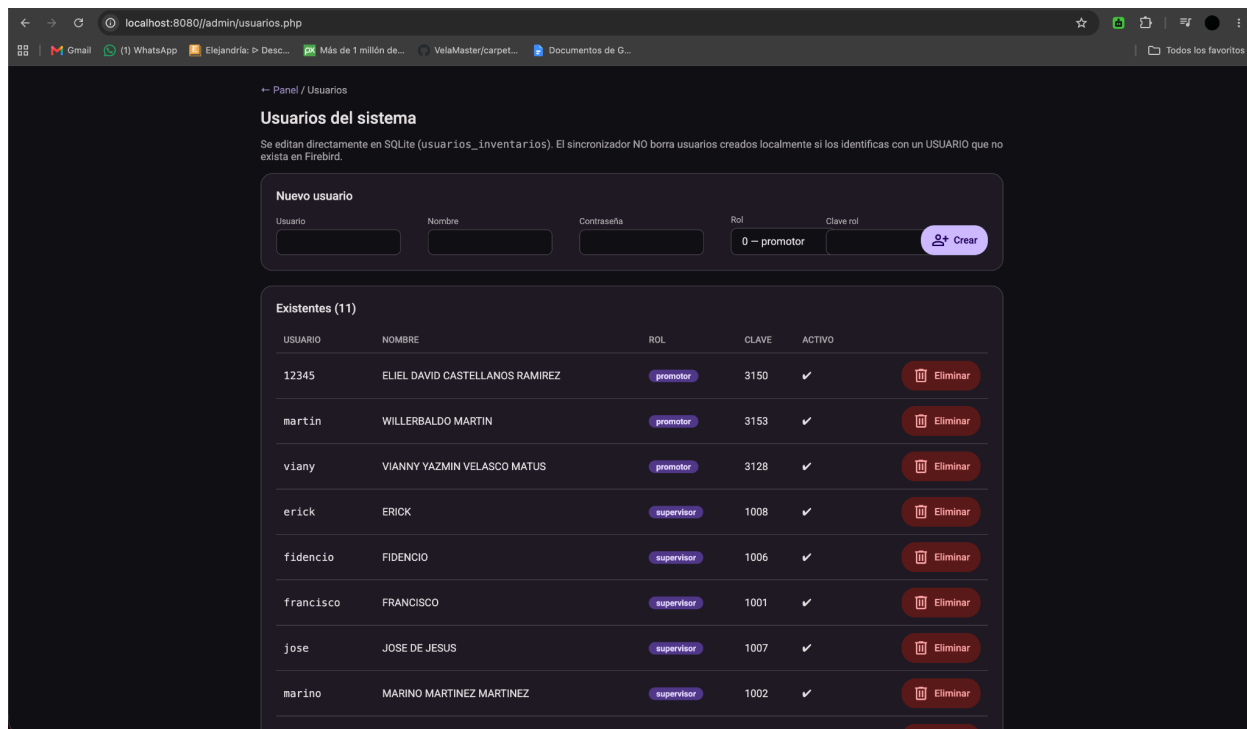
http://172.24.10.251:8090/admin/?key=<TOKEN_SECRETO>

⚠ Advertencia de seguridad: El valor real del token nunca debe escribirse en este manual, en la memoria del proyecto, ni en ningún documento que pueda hacerse público (incluyendo el repositorio de código si es accesible por terceros). Quien tenga ese token tiene acceso administrativo completo al sistema. El token debe rotarse periódicamente y comunicarse únicamente por un canal seguro al personal autorizado.

Una vez dentro del panel con el token vigente, el alta de un nuevo usuario sigue el flujo estándar del formulario de `usuarios.php`: nombre, rol asignado y una contraseña temporal que el propio usuario deberá cambiar en su primer inicio de sesión.

The screenshot displays the 'Panel Administrador' interface. At the top, there are navigation buttons: 'Sincronizar BDD', 'Configurar Firebird', 'Usuarios', 'Conciliar OPE', and 'Visor de errores'. Below these is a 'TOKEN DE ACCESO REMOTO' section with a text field containing a long alphanumeric string, a 'Copiar' button, and a 'Regenerar' button. A note indicates that users can access the system via a URL like `https://inventariosliconsaoaxaca.duckdns.org/admin/?key=...`. The 'Estado de SQLite' section shows four metrics: 'LECHERIAS' (1042), 'USUARIOS' (11), 'MUNICIPIOS' (570), and 'LOCALIDADES' (13092). Below this is an 'ERRORES HOY' section showing '0' errors. The 'Últimas sincronizaciones' section contains a table with the following data:

TABLA	FILAS	ESTADO	FECHA
inventarios_mensuales	24	OK	2026-06-18 16:50:31
mapeo_supervisor_lecheria	706	OK	2026-06-18 16:50:31
lecheria	1042	OK	2026-06-18 16:50:31
localidad	13092	OK	2026-06-18 16:50:31
municipio	570	OK	2026-06-18 16:50:31



Figuras 4 y 5: Panel de administrador.

Nota. La clave rol es su número de nómina.

4.3 Contraseñas

Un usuario puede cambiar su propia contraseña desde `cambiar_contraseña.php`. El sistema nunca almacena ni muestra la contraseña en texto plano, ya que se guarda mediante una función de hash. Si un usuario olvida su contraseña, el administrador puede asignarle una nueva contraseña temporal desde el panel de administración descrito en la sección 4.2.

5. Despliegue en la red institucional

5.1 Infraestructura (Windows Server 2012, VMware, Ubuntu Server)

El sistema corre actualmente sobre un entorno de virtualización dentro de la institución, no sobre un proveedor de nube.

- **Servidor físico:** Windows Server 2012, con dirección IP institucional 172.24.10.251.

- **Virtualización:** VMware Workstation, instalado sobre el Windows Server.
- **Máquina virtual:** Ubuntu Server, con su adaptador de red configurado en modo NAT. Usuario/Contraseña = ubuntu/facilita
- **Contenedor:** Docker, ejecutándose dentro de la máquina virtual, donde corre la aplicación completa.

5.2 Configuración de red (NAT y port forwarding)

El acceso al sistema se resuelve mediante una regla de port forwarding configurada en VMware Workstation (Edit > Virtual Network Editor > NAT Settings), que redirige el puerto 8090 del servidor físico hacia el puerto correspondiente dentro de la máquina virtual. Esto permite que cualquier equipo dentro de la red institucional acceda al sistema mediante:

```
http://172.24.10.251:8090
```

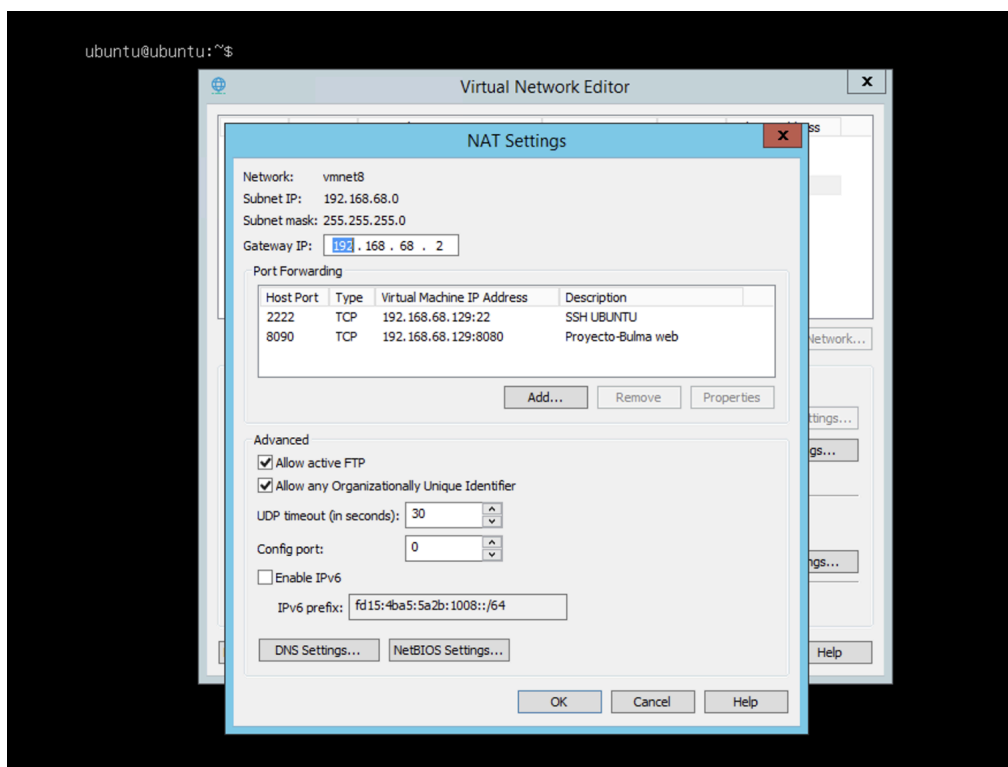


Figura 6: Configuración de NAT.

El sistema permanece dentro de la red institucional y no se expone a internet; el acceso queda restringido a los equipos conectados a esa misma red local.

Despliegue institucional en red local

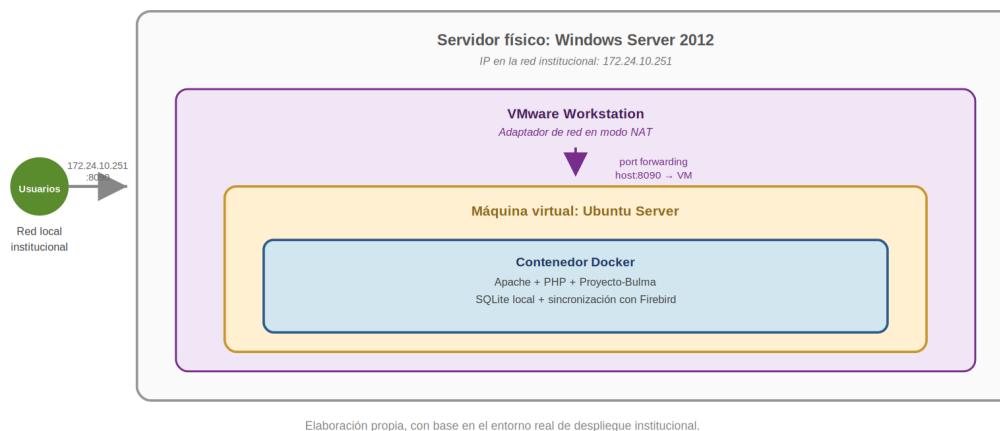


Figura 7. Despliegue institucional en red local.

6. Instalación del sistema

6.1 Requisitos previos

Antes de instalar el sistema dentro de la máquina virtual, verifique que se cumplan los siguientes requisitos:

Recurso	Mínimo recomendado	Notas
vCPU	2	Suficiente para el volumen actual de usuarios concurrentes.
Memoria RAM	1.5 GB	Ampliar si se incrementa el número de usuarios simultáneos.
Almacenamiento	15 GB	Incluye el sistema operativo, el contenedor, la base SQLite y los documentos generados.

Software	Versión recomendada	Uso
Docker / Docker Compose	20.x o superior	Construcción y ejecución del contenedor de la aplicación.

PHP	8.x (dentro del contenedor)	Ejecución del backend del sistema.
Driver Jaybird (JDBC)	jaybird-jdk18-3.0.12	Comunicación con la base de datos Firebird heredada.
Git	Cualquier versión reciente	Obtención y actualización del código fuente.

6.2 Clonar el repositorio (Git)

Dentro de la máquina virtual Ubuntu Server, clone el repositorio del proyecto:

```
git clone https://github.com/VelaMaster/Proyecto-Bulma.git
```

Advertencia: Este proyecto utiliza transferencia manual de archivos (scp/rsync) como práctica preferida para configuración sensible, precisamente para evitar exponer credenciales o el token de administración en el historial de Git.

6.3 Configuración del entorno

Configure `admin/config.php` con los parámetros del entorno: rutas de la base de datos local, credenciales de conexión a Firebird, y el token de acceso al panel de administración descrito en la sección 4.2.

← Panel / Configurar Firebird

Configuración de Firebird

El sincronizador usa estos datos para leer la BDD origen. La contraseña se guarda en texto plano en SQLite local (acceso restringido por IP).

✓ Conexión exitosa (95 ms)

HOST / IP	PUERTO
db	3050
USUARIO	CONTRASEÑA
SYSDBA	masterkey
RUTA DE LA BDD (EN EL SERVIDOR FIREBIRD)	
/firebird/data/DB_SIDISTLOCAL.FDB	
CHARSET	
NONE	

Guardar Guardar y probar conexión

Figura 8: Configuración de entorno.

6.4 Construir y levantar con Docker

Construya y levante el contenedor con Docker Compose:

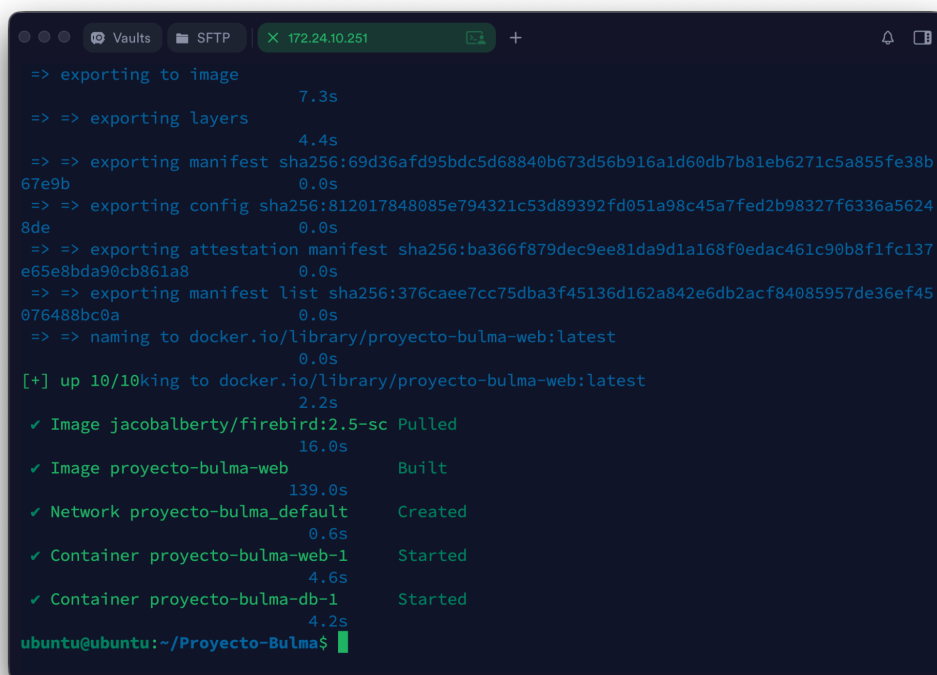
```
docker compose up -d --build
```

Este comando ejecuta `entrypoint.sh`, que prepara el entorno PHP y los drivers necesarios para Firebird antes de iniciar el servidor web dentro del contenedor.

6.5 Verificación de la instalación

```
docker ps
```

Acceda a `http://172.24.10.251:8090` desde un navegador en la red institucional y confirme que la pantalla de inicio de sesión se muestra correctamente para los cuatro módulos.



```
=> exporting to image
7.3s
=> => exporting layers
4.4s
=> => exporting manifest sha256:69d36afd95bdc5d68840b673d56b916a1d60db7b81eb6271c5a855fe38b
67e9b 0.0s
=> => exporting config sha256:812017848085e794321c53d89392fd051a98c45a7fed2b98327f6336a5624
8de 0.0s
=> => exporting attestation manifest sha256:ba366f879dec9ee81da9d1a168f0edac461c90b8f1fc137
e65e8bda90cb861a8 0.0s
=> => exporting manifest list sha256:376cae7cc75dba3f45136d162a842e6db2acf84085957de36ef45
076488bc0a 0.0s
=> => naming to docker.io/library/proyecto-bulma-web:latest
0.0s
[+] up 10/10king to docker.io/library/proyecto-bulma-web:latest
2.2s
✓ Image jacobalberty/firebird:2.5-sc Pulled
16.0s
✓ Image proyecto-bulma-web Built
139.0s
✓ Network proyecto-bulma_default Created
0.6s
✓ Container proyecto-bulma-web-1 Started
4.6s
✓ Container proyecto-bulma-db-1 Started
4.2s
ubuntu@ubuntu:~/Proyecto-Bulma$
```

Figura 9. Listado con `docker ps`.

7. Mantenimiento

7.1 Monitoreo y bitácora de errores

El sistema cuenta con un servicio de bitácora, `src/Servicio/LoggerErrores.php`, visible para el administrador a través de `admin/errores.php` (API: `admin/api_errores.php`). Revise esta bitácora periódicamente, especialmente tras actualizaciones o cambios en el padrón sincronizado desde Firebird.

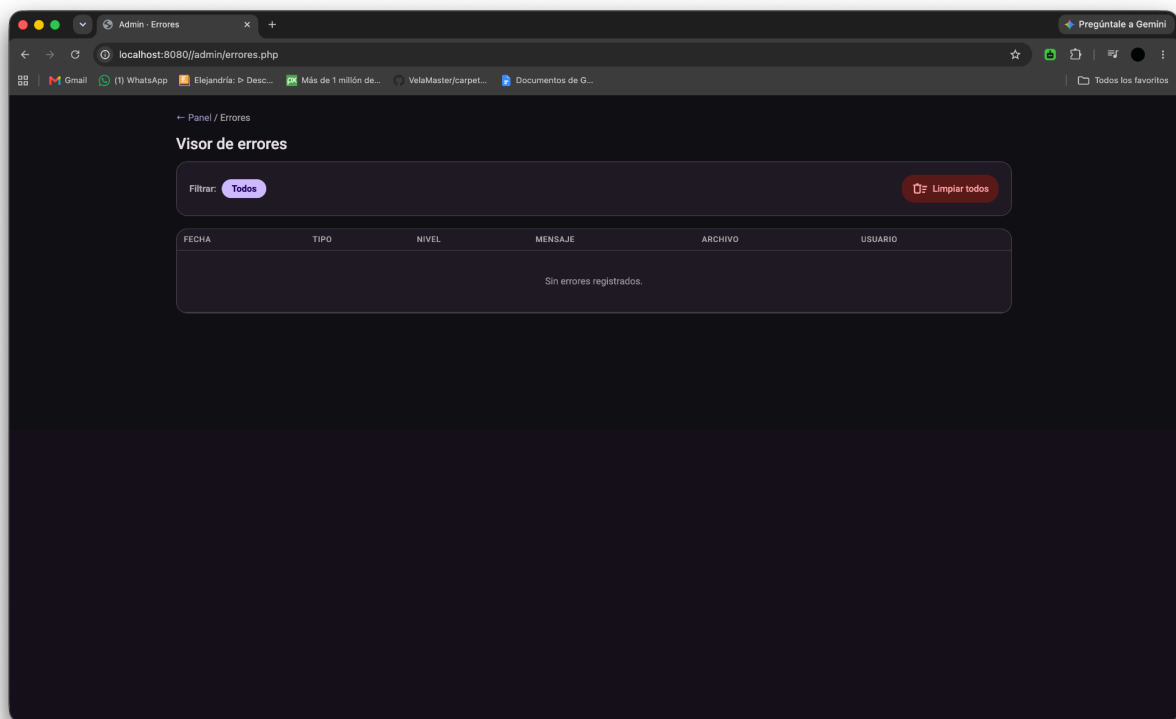


Figura 10 Visor de errores.

7.2 Cómo aplicar actualizaciones

Para aplicar una actualización del código fuente dentro de la máquina virtual:

```
git pull origin maindocker compose up -d --build
```

Advertencia: Respalde siempre `datos/local.db` antes de actualizar (ver sección 3.3), en caso de que la nueva versión incluya cambios en la estructura de la base de datos.

7.3 Respaldos periódicos

Además del respaldo de la base de datos, respalde periódicamente la carpeta completa de documentos generados (datos/promotores, datos/distribución y datos/supervisor), ya que contiene los comprobantes PDF y Excel entregados a los usuarios y a Diconsa.

7.4 Solución de problemas comunes

Problema	Causa probable	Acción recomendada
El sistema no carga tras una actualización.	El contenedor no terminó de construirse correctamente.	Revisar con docker compose logs y reconstruir con docker compose up -d --build.
Errores de “base de datos ocupada”	Múltiples escrituras simultáneas sobre SQLite.	Verificar el volumen de usuarios concurrentes; si persiste, evaluar la migración descrita en la memoria técnica.
El padrón de beneficiarios no se actualiza.	Falla en la sincronización con Firebird.	Revisar la bitácora de errores y la conexión del driver Jaybird.
No se puede acceder al panel de administración.	Token vencido, rotado o mal copiado en la URL.	Verificar el token vigente en admin/config.php; nunca compartirlo por canales no seguros.
Un usuario no puede generar su PDF.	Error en FPDF o datos incompletos del formulario.	Revisar la bitácora de errores para identificar el mensaje devuelto por FPDF.

8. Anexos

8.1 Estructura de carpetas del proyecto

Carpeta / archivo	Contenido
admin/	Módulo de administración: gestión de usuarios, errores y sincronización.
distribucion/	Módulo del Departamento de Distribución.
promotores/	Módulo del actor promotor.

supervisor/	Módulo del actor Supervisor.
src/Controlador, src/Servicio, src/Repositorio	Lógica de negocio y acceso a datos del sistema.
database/	Base de datos Firebird heredada y su driver de conexión.
datos/	Base de datos SQLite local y documentos generados por cada módulo.
fpdf/	Librería de generación de documentos PDF.
Dockerfile, docker-compose.yml, entrypoint.sh	Configuración de contenerización y despliegue.

Cierre:

Este manual técnico documenta el estado del sistema al momento de su entrega, correspondiente a la versión 1.0 desplegada en la red institucional de Leche del Bienestar en Oaxaca. Su valor no termina con la entrega del proyecto: conforme el sistema crezca, se le agreguen funciones, cambie de servidor o se ajuste su infraestructura, este documento debe actualizarse en la misma medida, de modo que siga siendo un reflejo fiel de cómo funciona el sistema y no una fotografía desactualizada de cómo funcionaba en su primera versión.

Se recomienda que cualquier persona que asuma la responsabilidad técnica del sistema en el futuro revise primero este manual antes de modificar la arquitectura, la base de datos o el entorno de despliegue, y que documente aquí mismo cualquier cambio relevante que realice.

Finalmente, vale la pena recordar que las decisiones documentadas aquí, la persistencia híbrida entre SQLite y Firebird, el control de acceso basado en roles, el despliegue contenerizado dentro de una máquina virtual institucional, no son arbitrarias: responden a los hallazgos y a la justificación presentados en los capítulos previos de la memoria técnica del proyecto. Mantener esa coherencia entre la teoría que sustenta al sistema y la práctica que lo opera.